NIVEL 2

RECURSOS AVANZADOS DEL LENGUAJE PYTHON

Daxus
LATAM

MÓDULO 1: **GESTORES DE PAQUETES Y ENTORNOS VIRTUALES EN PYTHON**

En este módulo aprenderás cómo manejar las **librerías o paquetes** que usa Python para agregar nuevas funciones a tus proyectos. Primero conocerás **PIP**, una herramienta que viene instalada automáticamente con Python. Con PIP puedes:

- Instalar nuevas librerías (por ejemplo, para trabajar con datos, gráficos o inteligencia artificial).
- Actualizar las que ya tienes.
- Eliminar las que ya no necesitas.

También aprenderás a **verificar la versión de PIP** que tienes, **actualizarla** si es necesario y usarla correctamente desde la terminal. Después, verás los **entornos virtuales** (virtual environments), que sirven para que cada proyecto tenga sus propias librerías y versiones, **sin afectar a otros proyectos.**

Esto es muy útil cuando trabajas en varios proyectos distintos o con diferentes equipos. Te mostraré cómo **crear, activar y configurar** un entorno virtual desde **Visual Studio Code**, paso a paso.

Por último, aprenderás a **compartir tus dependencias** (las librerías que usa tu proyecto) usando un archivo llamado **requirements.txt**. Este archivo facilita que otras personas puedan instalar exactamente las mismas librerías que tú con un solo comando.

También conocerás **Poetry**, una herramienta más moderna que hace todo este proceso de forma automática: gestiona dependencias, crea proyectos y configura los entornos virtuales por ti.

Continúa en la próxima página

PIP

→ VERIFICACIÓN DE VERSIONES

Contexto: Confirmar que Python y PIP están instalados.

Desarrollo en la terminal

```
python --version
pip --version
```

¿Para qué sirve esta caja?

En esta sección encontrarás comandos y fragmentos de código listos para copiar y pegar.

→ ACTUALIZAR PIP

Contexto: Instalar la versión más reciente del gestor (mira la versión más reciente en: https://pypi.org/project/pip/.

Desarrollo en la terminal

```
python -m pip install --upgrade pip
```

→ INSTALAR Y DESINSTALAR LIBRERÍAS

Contexto: Instalación de librerías y probar instalación de una librería llamada requests.

Desarrollo en la terminal

```
pip install nombre_libreria
pip uninstall nombre_libreria
```

EJEMPLO:

Contexto: Instalación de librerías y probar instalación de una librería llamada requests.

Desarrollo en la terminal

```
pip install nombre_libreria
pip uninstall nombre_libreria
```

POETRY

→ INSTALAR POETRY

Desarrollo en la terminal

```
pip install poetry
```

→ CONFIGURAR POETRY

Desarrollo en la terminal

```
poetry config --list
```

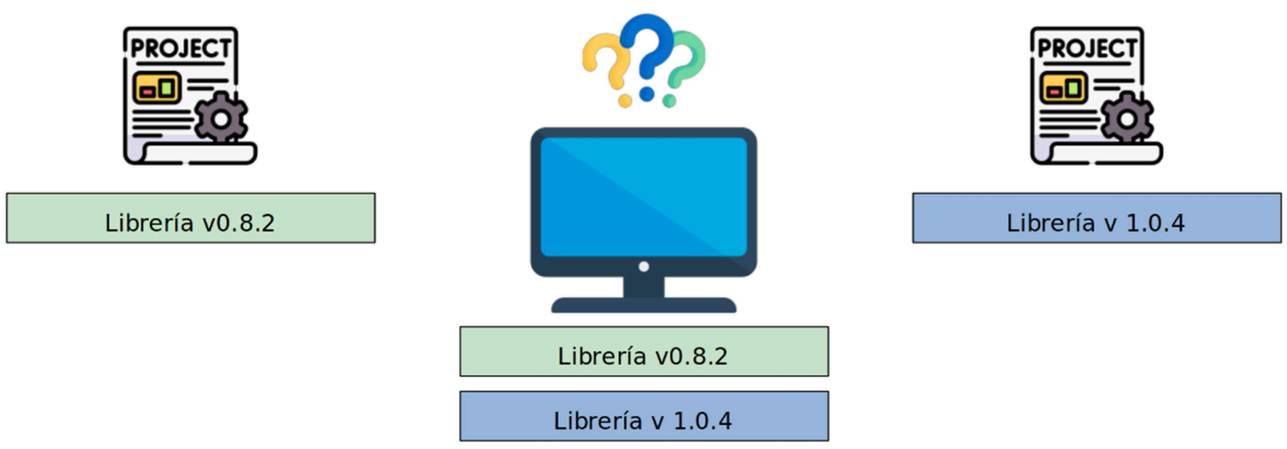
```
virtualenvs.in-project = null
```

- virtualenvs.in-project = true → Poetry crea el venv **dentro de la carpeta del proyecto**, en algo tipo .venv/.
- virtualenvs.in-project = false → Poetry crea el venv en una **carpeta global** (por defecto en la ruta de caché de Poetry).
- virtualenvs.in-project = null → significa “usa el valor por defecto” (que normalmente es false), o sea: **no en el proyecto**, sino en la ruta global.

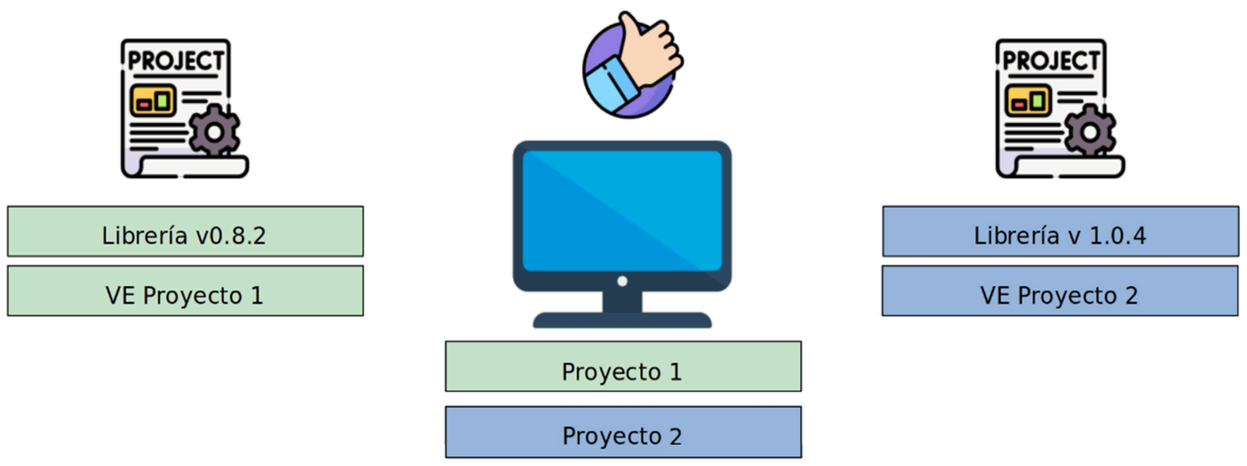
CASOS DE ESTUDIO REALES

→ CASO 1: CONFLICTOS ENTRE PROYECTOS

Situación: Dos proyectos usan versiones diferentes de una misma librería.
Análisis: Instalar librerías de manera global puede generar errores por incompatibilidad.



Lecciones aprendidas: Usar entornos virtuales garantiza independencia entre proyectos y evita conflicto



PROCESO: CREAR Y CONFIGURAR UN ENTORNO VIRTUAL

→ PIP

Objetivo: Aislar dependencias de un proyecto Python.

Pasos:

1. Crear carpeta del proyecto.
2. Abrirla con **Visual Studio Code.**
3. Abrir terminal **(Ctrl + Shift + ñ).**
4. Ejecutar:

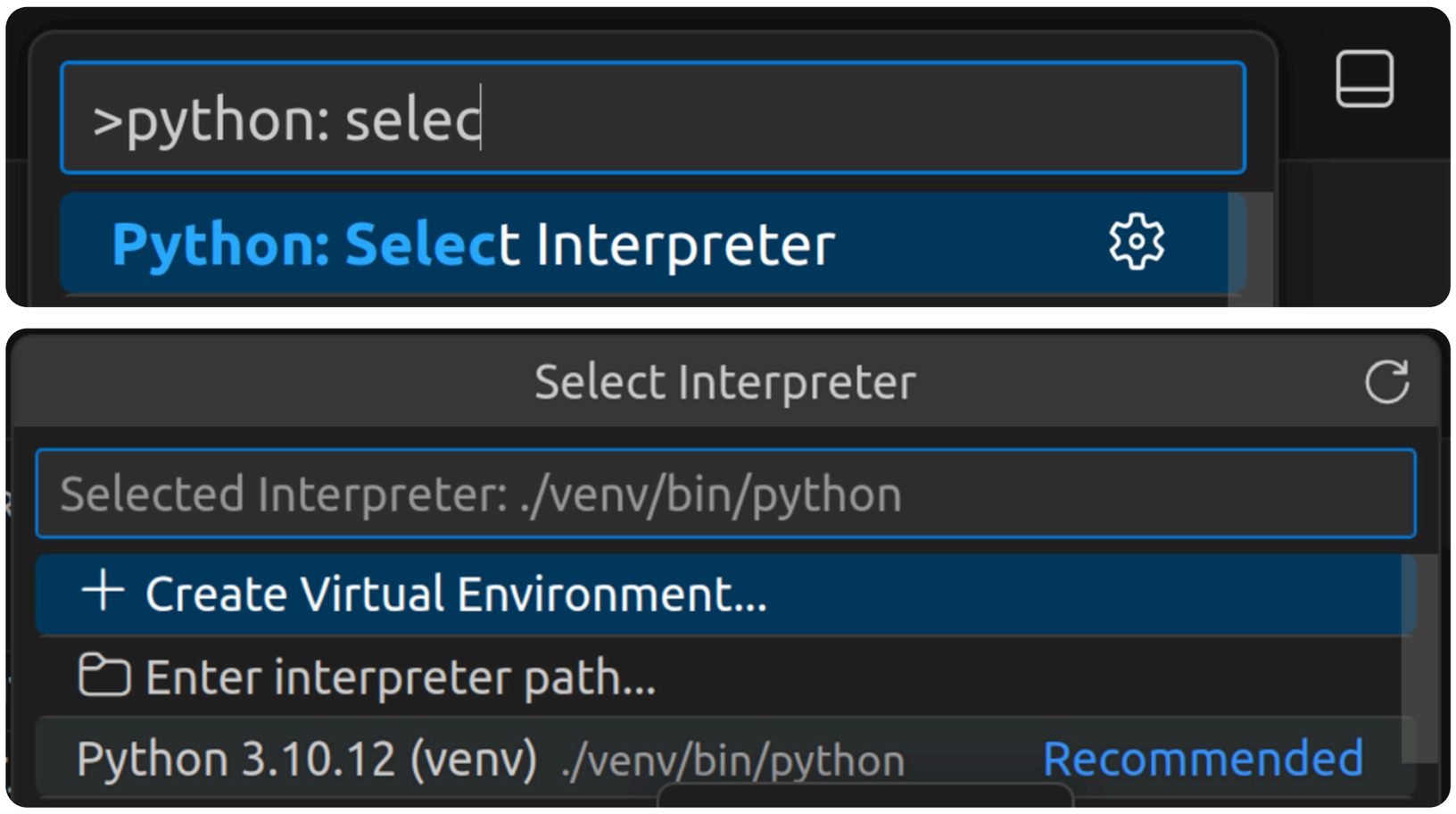
```
python -m venv venv
```

5. Activar entorno:

```
venv\Scripts\activate
```

Asegurese de que el entorno virtual está activo en la terminal, debe salir “(nombre_entorno_virtual)”

6. Configurar intérprete en VS Code: Ctrl + Shift + P → *Python: Select Interpreter* → Seleccionar el entorno “venv”.



7 Instalar librerías necesarias.

Verificación: El prompt muestra (venv) antes del nombre del directorio.

→ POETRY

Objetivo: Aislar dependencias de un proyecto Python.

Pasos:

8 Crear carpeta del proyecto con el comando en la terminal:

```
poetry new nombre_proyecto
```

9 Abrirla con **Visual Studio Code.**

10 Abrir terminal **(Ctrl + Shift + ñ).**

11 Ejecutar en la terminal:

```
poetry env activate
```

12 Activar entorno: Si cerramos y abrimos la terminal se abre poetry.

Desarrollo en la terminal

```
poetry add nombre_libreria
Poetry remove nombre_librería
```

EJEMPLO:

Desarrollo

```
poetry add colorama
poetry add pandas
poetry remove pandas
```

PROCEDIMIENTO: COMPARTIR UN PROYECTO CON DEPENDENCIAS

→ PIP

Prerrequisitos: Entorno virtual activo y dependencias instaladas.
Secuencia:

1. Generar archivo de dependencias en la terminal:

```
pip freeze > requirements.txt
```

2. Compartir el proyecto y el archivo requirements.txt (OJO: sin la carpeta del entorno virtuel, ej: venv).
3. El colaborador crea su entorno virtual.
4. Instala las dependencias con el comando ela terminal:

```
pip install -r requirements.txt
```

Puntos de control: Confirmar que las librerías aparecen con el comando: pip freeze.

→ POETRY

Prerrequisitos: Entorno virtual activo y dependencias instaladas.
Secuencia:

5. Comprimir carpeta sin el entorno virtual (carpeta .venv)t
6. Compartir el proyecto comprimido
7. El colaborador crea su proyecto
8. Instala las dependencias con: poetry install

Puntos de control: Confirmar que las librerías aparecen con poetry show.

PROCEDIMIENTO: DESACTIVAR ENTORNO VIRTUAL

→ **PIP Y POETRY**

Prerrequisitos: Entorno virtual activo y escribir en la terminal:

```
deactivate
```

ADVERTENCIA: Instalar librerías globalmente puede causar conflictos entre proyectos.

▶ **Consecuencia:** Incompatibilidades y errores de ejecución.

▶ **Prevención:** Usar entornos virtuales aislados.

MÓDULO 2: **COMPRENSIONES, FUNCIONES LAMBDA Y FUNCIONES DE ORDEN SUPERIOR EN PYTHON**

En este módulo aprenderás algunas herramientas **avanzadas pero muy útiles** de Python que te ayudarán a escribir código **más corto, limpio y fácil de leer**. Primero conocerás las **comprensiones de listas y diccionarios**. Estas te permiten crear listas o diccionarios **en una sola línea de código**, sin necesidad de usar varios bucles o condiciones largas.

Por ejemplo, puedes generar una lista de números pares del 1 al 10 con una sola línea, ¡en lugar de escribir varias! Después aprenderás sobre las **funciones lambda**, que son **funciones pequeñas y sin nombre**. Sirven para realizar tareas rápidas como sumar, multiplicar o aplicar una condición, sin tener que usar la palabra **def** para definir una función completa.

Son muy útiles cuando necesitas una función sencilla dentro de otra instrucción. Luego veremos las funciones **map()**, **filter()**, **reduce() y sorted()**, que son llamadas **funciones de orden superior**. Estas funciones pueden **recibir otras funciones como parámetro**, lo que te permite transformar, filtrar, combinar u ordenar listas de datos con pocas líneas de código. Por ejemplo:

- **map()** sirve para aplicar una operación a todos los elementos de una lista.
- **filter()** te ayuda a quedarte solo con los elementos que cumplen una condición.
- **reduce()** combina los elementos hasta obtener un único resultado (como una suma total).
- **sorted()** te permite ordenar listas o tuplas fácilmente.

Finalmente, vas a ver ejemplos prácticos donde se combinan todos estos conceptos. Con ellos aprenderás a escribir código que se ve más profesional, es decir, siguiendo el estilo simple, claro y elegante que caracteriza a Python.

→ LIST COMPREHENSION BÁSICA

Contexto: Crear una lista con los números del 1 al 10 en una sola línea.

Sintaxis Básica: [nueva_expresión for elemento in iterable]

Desarrollo usando un for

Entrada

```
numeros = [ ]
for i in range(1, 11):
     numeros.append(i)
print(numeros)
```

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Desarrollo usando list comprenhension

Entrada

```
numeros = [i for i in range(1, 11)]
print(numeros)
```

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

→ LIST COMPREHENSION CON CONDICIÓN

Contexto: Crear una lista con números pares del 1 al 10.

Sintaxis: [nueva_expresión for elemento in iterable if condición]

Desarrollo con list comprenhension

Entrada

```
pares = [i for i in range(1, 11) if i % 2 == 0]
print(pares)
```

Salida

```
[2, 4, 6, 8, 10]
```

→ DICT COMPREHENSION — INVERTIR LLAVES Y VALORES

Sintaxis: {clave: valor for elemento in iterable}

Desarrollo con dict comprenhension

Entrada

```
original = {'a': 1, 'b': 2, 'c': 3}
invertido = {v: k for k, v in original.items()}
print(invertido)
```

Salida

```
{1: 'a', 2: 'b', 3: 'c'}
```

→ DICT COMPREHENSION CON CONDICIONAL – CLASIFICAR NÚMEROS PARES E IMPARES

Sintaxis: {clave: valor if condición else not_condición for elemento in iterable}

Entrada

```
clasificacion = {x: "par" if x % 2 == 0 else "impar" for x in
range(5)}
print(clasificacion)
```

Salida

```
{0: 'par', 1: 'impar', 2: 'par', 3: 'impar', 4: 'par'}
```

→ FUNCIÓN LAMBDA BÁSICA - SUMAR DOS NÚMEROS

Sintaxis: lambda argumentos: expresión

Entrada

```
sumar = lambda a, b: a + b
print(sumar(5, 18))
```

Salida

```
23
```

Desarrollo análogo con def

Entrada

```
def sumar(a, b):
return a+b
print(sumar(5, 2))
```

Salida

```
7
```

ADVERTENCIA: Las funciones lambda no deben usarse para lógicas complejas.

▶ **Consecuencia:** Código difícil de leer y mantener.

▶ **Prevención:** Usar **def** cuando se requieran múltiples líneas o condiciones anidadas.

Continúa en la próxima página

→ LAMBDA CON ARGUMENTOS VARIABLES

Entrada

```
sumar_todo = lambda *args: sum(args)
print(sumar_todo(1, 6, 98))
```

Salida

```
105
```

→ MAP() CON FUNCIÓN LAMBDA

Sintaxis: map(función, iterable)

Contexto: Elevar al cubo cada número en una lista de strings.

Entrada

```
numeros = ['1', '2', '3', '4']
cubos = list(map(lambda n: int(n) ** 3, numeros))
```

Salida

```
[1, 8, 27, 64]
```

→ MAP() CON FUNCIÓN LAMBDA

Sintaxis: filter(función, iterable)

Entrada

```
lista = list(range(1, 101))
pares = list(filter(lambda x: x % 2 == 0, lista))
```

Salida

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36,
38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70,
72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98, 100]
```

 Todos los derechos reservados

→ FILTER() CON APLICACIÓN PRÁCTICA

Contexto: Filtrar notas mayores al promedio.

Entrada

```
from statistics import mean
notas = [2.3, 3.4, 5.0, 2.8, 4.5]
media = mean(notas)
mejores = list(filter(lambda n: n > media, notas))
```

Salida

```
[3.4, 5, 4.5, 4.3]
```

→ REDUCE() ACUMULANDO SUMAS

Sintaxis:

from functools import reduce

reduce(función, iterable, [valor_inicial])

Entrada

```
from functools import reduce
numeros = [1, 2, 3, 4, 5]
reduce(lambda x, y: x + y, lista)
```

Salida

```
15
```

→ PROCEDIMIENTO: REDUCIR UNA LISTA CON SORTED()

Sintaxis: sorted(iterable, key=None, reverse=False)

Entrada

```
nombres_edades = [("Nicolas", 32), ("Laura", 35), ("Felipe", 36),
("Esteban", 30), ("Miguel", 60)]
print(sorted(nombres_edades, key=lambda persona: persona[1]))
```

 Todos los derechos reservados

Salida

```
[('Esteban', 30), ('Nicolas', 32), ('Laura', 35), ('Felipe', 36),
('Miguel', 60)]
```

Entrada

```
print(sorted(nombres_edades, key=lambda persona: persona[1],
reverse=True))
```

Salida

```
[('Miguel', 60), ('Felipe', 36), ('Laura', 35), ('Nicolas', 32),
('Esteban', 30)]
```

EJERCICIOS

01 **List Comprehension - Cuadrados**
Crea una lista con los cuadrados de los números del 1 al 10 usando *list comprehension*.

02 **Dict Comprehension - Contar letras**
Dada una cadena "hello", crea un diccionario donde las claves sean las letras y los valores sean el número de veces que aparece cada letra, usando *dict*

03 **Lambda - Ordenar por último carácter**
Dada la lista ["manzana", "pera", "kiwi", "uva"], ordénala usando sorted() y una *función lambda* según el último carácter de cada palabra.

04 **map() - Convertir a mayúsculas**
Dada una lista de palabras ["python", "lambda", "map"], usa map() para convertir cada palabra a mayúsculas.

05 **filter() - Números pares**
Dada una lista de números del 1 al 20, usa filter() con una función lambda para obtener solo los números pares.

06 **reduce() - Producto de elementos**
Dada la lista [2, 3, 4], usa reduce() (del módulo functools) para calcular el producto de todos sus elementos.

07 **sorted() - Ordenar por longitud**
Ordena la lista ["zorro", "oso", "elefante", "gato"] por la longitud de las palabras, de menor a mayor.

08 **List Comprehension con condición**
Crea una lista con los números del 1 al 20 que sean divisibles entre 3, usando *list comprehension*.

09 **map() y lambda - Aplicar operación**
Dada una lista [1, 2, 3, 4], usa map() con una función lambda para devolver una lista donde cada elemento sea elevado al cubo y luego se le sume 10.

10 **filter() y lambda - Palabras que empiezan con vocal**
Dada la lista ["apple", "banana", "orange", "egg", "ice"], filtra aquellas palabras que empiecen con una vocal usando filter() y *lambda*.

MÓDULO 3: **FECHAS, HORAS Y TIEMPOS EN PYTHON**

En este módulo aprenderás a trabajar con fechas y horas en Python de manera fácil y precisa, usando una herramienta muy útil llamada módulo datetime. Primero conocerás las clases principales:

- **date**, que sirve para manejar solo fechas (como año, mes y día).
- **datetime**, que permite trabajar con **fechas y horas (horas, minutos, segundos y microsegundos)**

Aprenderás a **crear tus propias fechas y horas**, a **darles formato** (por ejemplo, mostrar el día primero o el mes primero) y a **acceder a partes específicas**, como el año, el mes o el día. Luego verás algunos **métodos muy útiles** del módulo **datetime**, como:

- **today()**, que devuelve la fecha actual.
- **weekday()**, que te indica qué día de la semana es (lunes, martes, etc.).
- **time()**, que obtiene la hora exacta.
- **now()**, que muestra la fecha y la hora actuales.

También aprenderás sobre la clase **timedelta**, que se usa para **sumar o restar días y horas**, o para calcular **cuánto tiempo ha pasado entre dos fechas**. Además, conocerás la librería **pytz**, que permite trabajar con **zonas horarias** (por ejemplo, ajustar la hora de Bogotá, Madrid o Nueva York). Esto es muy importante si tus proyectos manejan datos internacionales o necesitan precisión en el tiempo.

Finalmente, aprenderás a **convertir fechas en texto y viceversa** con los métodos **strptime()** y **strftime().** Estas funciones son muy útiles para leer fechas que vienen en formato texto (por ejemplo, desde una base de datos o un archivo) o para mostrar la fecha al usuario con el formato que tú elijas.

En resumen, después de este módulo podrás crear, modificar, comparar y mostrar fechas y horas de manera profesional, una habilidad fundamental para cualquier desarrollador.

 Todos los derechos reservados

→ CREAR UNA FECHA CON DATE

Entrada

```
from datetime import date
fecha = date(2025, 8, 25)
print(fecha)
```

Salida

```
2025-08-25
```

→ ACCEDER A PARTES DE UNA FECHA

Entrada

```
from datetime import date
fecha = date(2025, 1, 1)
print(fecha.year, fecha.month, fecha.day)
```

Salida

```
2025  1  1
```

→ FECHA ACTUAL CON TODAY()

Entrada

```
from datetime import date
fecha_actual = date.today()
print(fecha_actual)
```

Salida

```
(este cambia según el día que lo compiles): 2025-11-14
```

→ DÍA DE LA SEMANA

A el día de la semana se le asocia un vaor número, para cada día es:

0 - Lunes
1 - Martes
2 - Miércoles
3 - Jueves
4 - Viernes
5 - Sábado
6 - Domingo

Entrada

```
from datetime import date
print(date.today().weekday())
```

Salida

```
(EJ: SI ES LUNES)
0
```

→ CREAR FECHA Y HORA CON DATETIME

Entrada

```
from datetime import datetime
datetime(year=2025, month=8, day=25, hour=15, minute=35, second=34,
microsecond=0)
```

Salida

```
datetime.datetime(2025, 8, 25, 15, 35, 34)
```

Aunque no es necesario colocar los argumentos, pero si en el orden de año-mes-día-hora-minuto-segundo-microsegundo, así

→ ACCESO A PARTES DEL DATETIME

Entrada

```
print(f"Año de Datetime: {date_time.year}")
print(f"Mes Datetime: {date_time.month}")
print(f"Día Datetime: {date_time.day}")
print(f"Hora Datetime: {date_time.hour}")
```

```
print(f"Minuto(s) Datetime: {date_time.minute}")
print(f"Segundo(s) Datetime: {date_time.second}")
```

Salida

```
Año de Datetime: 2025
Mes Datetime: 8
Día Datetime: 25
Hora Datetime: 15
Minuto(s) Datetime: 35
Segundo(s) Datetime: 44
```

→ MÉTODOS DATE(), TIME(), NOW(), WEEKDAY()

Entrada

```
print(dt.date())   # Solo la fecha
print(dt.time())   # Solo la hora
print(datetime.now())  # Fecha y hora actuales
datetime.weekday(dt)  # Día de la semana en valor numérico
```

Salida

```
2025-08-25
2025-11-14 09:55:13.426030
15:35:44
0
```

→ ZONAS HORARIAS CON PYTZ

Entrada

```
import pytz
from datetime import datetime
```

Busque su zona horaria con alguna palabra clave como ‘Santiago’, ‘Buenos Aires’, ‘Bogota’ con el siguiente comando:

Entrada

```
list(filter(lambda x: "Bogota" in x, pytz.all_timezones))
```

Salida

```
# Result: ['America/Bogota']
```

Coloque el resultado usando los siguientes comandos:

```
timezone = pytz.timezone('America/Bogota')
```

Y consultamos el horario

Entrada

```
datetime.now().astimezone(timezone)
```

Salida

```
# Result: datetime.datetime(2025, 11, 16, 9, 55, 13, 474365,
tzinfo=<DstTzInfo 'America/Bogota' -05-1 day, 19:00:00 STD>)
```

→ DIFERENCIAS ENTRE FECHAS (TIMEDELTA)

Entrada

```
from datetime import date
fecha_inicial = date(2025, 1, 1)
fecha_final = date(2025, 3, 15)
diferencia = fecha_final - fecha_inicial


print(diferencia_dias)
print(type(diferencia_dias))
print()
print(diferencia_dias.days)
print(type(diferencia_dias.days))
```

Salida

```
73 days, 0:00:00
<class 'datetime.timedelta'>

73
<class 'int'>
```

→ SUMAR DÍAS U HORAS

Entrada

```
from datetime import datetime, timedelta
ahora = datetime.now()
nuevo = ahora + timedelta(days=1)
print(ahora)
print(nuevo)
```

Salida

```
2025-11-16 10:56:42.072273
2025-11-17 10:56:42.072273
```


Salida

EJERCICIOS

→ SISTEMA BANCARIO

Situación: Registrar la hora exacta de cada transacción.

Solución: Usar datetime.now() para registrar el instante exacto del movimiento.

Lección: Permite rastrear operaciones y detectar fraudes por tiempo.

→ APLICACIONES CON ZONAS HORARIAS

Situación: Un servidor global necesita mostrar horas locales.

Solución: Implementar pytz.timezone() para adaptar la hora según el país.

Lección: Mejora la coherencia temporal y evita errores en sistemas distribuidos.

→ TOKEN DE SEGURIDAD

Situación: Un código temporal debe expirar 15 minutos después de emitirse.

Solución: Crear un timedelta(minutes=15) y sumarlo al datetime.now().

Lección: Los deltas permiten controlar tiempos de expiración fácilmente.

→ CALCULAR LA DIFERENCIA ENTRE DOS FECHAS

Objetivo: Obtener el número de días entre dos fechas dadas y horas.

Entrada

```
fecha_inicial = date(2025,1,1)
fecha_final = date(2025,3,15)
diferencia_dias = fecha_final - fecha_inicial

print(diferencia_dias)
print(type(diferencia_dias))
print()
print(diferencia_dias.days)
print(type(diferencia_dias.days))
```

Salida

```
# Results:
73 days, 0:00:00
<class 'datetime.timedelta'>

 73
<class 'int'>
```

Note que la diferencia se da en días y horas, y el type en es de una clase **timedelta** y cuando accedemos al método **.dyas** y vemos su tipo accedemos a un entero que corresponden a los días. También podemos importar la clase **timedelta** par por ejemplo, sumar días.

EJEMPLO:

Entrada

```
from datetime import datetime, timedelta
suma_1_dia = timedelta( days=1)
print(suma_1_dia)
print(datetime.now().date())
print(datetime.now().date() + suma_1_dia)
```

Salida

```
1 day, 0:00:00
2025-11-17
2025-11-18
```

MANIPULACIÓN DE FECHAS CON STRPTIME() Y STRFTIME()

Puede revisar los códigos de formato en: https://docs.python.org/3/library/datetime.html#strftime-and-strptime-format-codes

→ CONVIRTIENDO STR A DATETIME CON STRPTIME()

Entrada

```
from datetime import datetime
datetime_str = "31/01/25 23:59:28"                    # es un str
print(datetime.strptime(datetime_str, "%d/%m/%y %H:%M:%S"))
print(datetime.strptime(datetime_str, "%d/%m/%y %H:%M:%S").date())
print(type(datetime.strptime(datetime_str, "%d/%m/%y %H:%M:%S")))
```

Salida

```
# Result:
2025-01-31 23:59:28
2025-01-31
<class 'datetime.datetime'>
```

→ CONVERTIENDO `DATETIME` A `STR` CON `STRFTIME()`

Entrada

```
fecha = datetime(2025,8,25,16,36,15)
print(f"La fecha y la hora son: {fecha.strftime("%A %d %B %Y, %I:
%M%p")}")
```

Salida

```
La fecha y la hora son: Monday 25 August 2025, 04:36PM
```

Ya que el resultado está en inglés para cambiearlo a español se puede usar locale
import **locale**

Entrada

```
import locale
locale.setlocale(locale.LC_ALL, "es_Es.UTF-8")
print(f"La fecha y la hora son: {fecha.strftime("%A %d %B %Y, %I:
%M%p")}")
```

Salida

```
La fecha y la hora son: Lunes 25 de Agosto del 2025, 04:36PM
```

EJERCICIOS

01 **datetime.date - Fecha actual**
Crea un objeto date con la fecha actual y muestra por pantalla el año, mes y día por separado.

02 **datetime.datetime - Fecha y hora actual**
Obtén la fecha y hora actual completa usando datetime y muéstrala en formato: "DD/MM/YYYY HH:MM:SS".

03 **timezone - Configurar zona horaria**
Crea un objeto datetime con zona horaria 'US/Eastern' y conviértelo a la zona horaria 'Europe/Madrid'.

04 **Cálculos con fechas - Diferencia entre fechas**
Calcula cuántos días han pasado desde tu fecha de nacimiento hasta hoy.

05 **Cálculos con timedelta - Sumar días**
Dada una fecha cualquiera, usa timedelta para sumarle 100 días y muestra la nueva fecha.

06 **Conversión string a fecha**
Convierte el string "15-03-2023" a un objeto datetime (formato: DD-MM-YYYY).

07 **Conversión fecha a string**
Toma la fecha y hora actual y conviértela a string con el formato: "Martes, 15 de Marzo de 2023".

08 **Día de la semana**
Dada una fecha específica, determina qué día de la semana fue (ej: "Lunes", "Martes", etc.).

09 **Comparación de fechas**
Crea dos fechas diferentes y determina cuál es más reciente, mostrando la diferencia en días.

10 **Edad exacta**
Calcula la edad exacta de una persona en años, meses y días dada su fecha de nacimiento.

 Todos los derechos reservados

MÓDULO 4: **LECTURA, ESCRITURA Y MANIPULACIÓN DE ARCHIVOS EN PYTHON**

En este módulo aprenderás a **trabajar con archivos en Python**, una habilidad muy útil cuando necesitas guardar información, leer datos o automatizar tareas. Comenzaremos con los **archivos de texto (.txt)**. Verás cómo **abrirlos, leerlos, escribir en ellos o agregar nueva información** sin borrar lo que ya tienen. También aprenderás qué significan los **modos de apertura:**

- **r** para leer,
- **w** para escribir (sobrescribe el contenido anterior),
- **a** para anexar (agregar texto al final).

Además, hablaremos de la **codificación de los archivos**, como **utf-8** o **latin-1**, que son los sistemas que permiten representar correctamente los caracteres especiales (como tildes o eñes). Luego conocerás dos formatos de datos muy usados:

- **CSV (Comma-Separated Values):** ideal para manejar tablas o datos que vienen de Excel o bases de datos.
- **JSON (JavaScript Object Notation):** un formato muy común para intercambiar datos entre aplicaciones o con APIs.

Aprenderás a **leer y escribir archivos CSV y JSON** en Python de forma práctica, y verás cómo estos formatos facilitan el trabajo con información estructurada y su compatibilidad con muchas herramientas externas. Después, se presenta el módulo os, que te permite interactuar con **el sistema operativo** desde Python. Con él podrás, por ejemplo, **crear y eliminar carpetas, listar archivos** o incluso **ejecutar comandos del sistema** sin salir del programa.

Finalmente, conocerás una de las bibliotecas más poderosas de Python: **pandas**. Aprenderás a **leer archivos grandes (como CSV), analizarlos fácilmente y mostrar los datos en forma de tabla**, lo que te servirá mucho si te interesa el análisis de datos o la automatización de reportes. En resumen, este módulo te enseñará a g**uardar, leer y manipular** información real, preparándote para trabajar con proyectos más grandes y profesionales.

 Todos los derechos reservados

→ LECTURA DE UN ARCHIVO TXT

Contexto: Leer un archivo llamado Biblia.txt

Entrada

```
archivo=open('Biblia.txt',mode='r',encoding='latin-1') # ‘r’
predeterminado
for linea in archivo:
     print(linea)
```

Salida

```
--texto en el archvi plano--
```

→ CERRAR ARCHIVO

Entrada

```
print(archivo.closed)
print(archivo.close())
print(archivo.closed)
```

Salida

```
False   (- Define el estado de apertura, en este caso el archivo
está abierto -)
None  (- Archivo cerrado -)
True (- Define el estado de apertura, en este caso el archivoestá
cerrado-)
```

→ LEER UNA SOLA LÍNEA

Entrada

```
archivo = open('Biblia.txt', encoding='latin1')
# Cada vez que se ejecuta lee la siguiente línea
print(archivo.readline())
archivo.close()
```

 Todos los derechos reservados

Salida

```
(- Muestra la primera líneao la siguinete, dependiendo de cuantas
veces se compile -)
```

→ LEER TODAS LAS LÍNEAS COMO LISTA

Entrada

```
archivo = open('Biblia.txt', encoding='latin-1')
texto = archivo.readlines()

print(type(texto))
texto
```

Salida

```
<class 'list'>
(- Lista con cada linea del txt como elemento -)
```

→ ESCRIBIR UN ARCHIVO DESDE CERO

Sintaxis: with open(nombre_archivo, modo, encoding, ...) as alias:

Entrada

```
archivo = open('mi_archivo.txt', 'w', encoding='utf-8')
frase = "Mi nombre es Tanjiro y estoy aprendiendo Python ⚔️"
archivo.write(frase)
archivo.close()
```

Salida

```
(- crea un archivo con el nombre ‘mi_archivo.txt’ con este contenido
-)
```

Continúa en la próxima página

→ AGREGAR TEXTO AL ARCHIVO

```
archivo = open('mi_archivo.txt', 'a', encoding='utf-8')
archivo.write("\nEstoy aprendiendo a manipular archivos
TXT") # Recuerde que “\n” es un salto de línea para que
no se escriba de corrido
archivo.close()
```

→ USO DEL CONTEXTO WITH

El uso del contexto “**with**” nos ayuda para que el archivo se cierre sin necesidad de escribir el comando “**.close**”

```
with open('mi_archivo.txt', 'a') as f:
    f.write("\nEscribiendo con contexto 'with'")
```

→ LECTURA DE UN CSV

- csv: **C**omma **S**eparated **V**alues (Valores Separados por Comas). Puede utilizar para como separador ";", "\t" (tabulación).
- Es un archivo de texto.
- El módulo csv nos permite trabajar con este tipo de archivos.
- Retorna un objeto `_csv.reader`.

Continúa en la próxima página

Entrada

```
import csv
with open('msft.csv', 'r') as f:  # Donde msft es el nombre del
archivo
    lector = csv.reader(f, delimiter=',') # El el 'delimiter' nos
permite decirle cual es la separación de nuestras columas
    for dato in lector:
    print(dato)
```

Salida

```
-- imprime cada uno de los resultados --
```

→ ESCRITURA DE UN CSV

Entrada

```
import csv
datos = [
    ["Nombre", "Edad", "Ciudad"],
    ["Juan", 28, "Madrid"],
    ["Ana", 22, "Barcelona"],
    ["Luis", 35, "Valencia"]
]
with open('personas.csv', mode='w', newline='\n') as archivo_csv:
# Crea un escritor de css
    escritor = csv.writer(archivo_csv)
# Escribir las filas en el css
    escritor.writerows(datos)

print("Archivo CSV creado con éxito.")
```

Salida

```
Archivo CSV creado con éxito. (- Y se crea un archvo con el nombre
correspondiente .csv -)
```

 Todos los derechos reservados

→ LEER JSON

- JSON: **J**avaScript **O**bect **N**otation.
- Es un archivo de texto.
- Formato de archivos compacto y **fácil de leer para las personas.**
- Comúnmente utilizado para intercambio de datos en la **web** y almacenamiento en bases de datos no relacionales (NoSQL).
- El módulo json nos permite trabajar con este tipo de archivos.
  - Retorna una list.

Entrada

```
import json

with open('mi_json.json', 'r') as f:
     data = json.read(f)
print(data)
print(type(data)
```

Salida

```
-- Nos da el diccionario con llaves y un tipo clase --
```

MANEJO DEL SISTEMA DE ARCHIVOS CON `OS`

Una de las principales ventajas de un lenguaje de programación de alto nivel como`Python` es que nos da una interfaz unificada para la manipulación del sistema de archivos desde cualquier sistema operativo, se trata de la librería base `os`.

Continúa en la próxima página

→ MÉTODO `GETCWD()`

Revisar el `directorio` actual

Entrada

```
import os
print(os.getcwd())
```

Salida

```
(- La ruta de donde se está ejecutando el script -)
```

→ MÉTODO `LISTDIR()`

El método listdir() sirve para listar el contenido de una carpeta (directorios y archivos) en Python.

Entrada

```
print(os.listdir("."))
```

Salida

```
(Muestra el contenido de la carpeta actual)
```

Entrada

```
print(os.listdir("./modulo_os/"))
```

Salida

```
(Muestra el contenido de una carpeta llamada “modulo_os” dentro de
la carpeta actual)
```

Continúa en la próxima página

→ MÉTODOS `JOIN()` Y `BASENAME()`

Revisar el `directorio` actual

Entrada

```
os.path.join(os.getcwd(), "nuevo_archivo.txt")
```

Una la rota completa con el nombre del archivo.

Entrada

```
os.path.basename(os.getcwd())
```

Salida

```
os.path.basename(os.getcwd())
# Result: se queda solo con el último pedacito de la ruta
```

→ MÉTODOS `ISFILE()` Y `ISDIR()`

Los métodos isfile() e isdir() sirven para verificar si una ruta corresponde a un archivo o a un directorio en Python.

Entrada

```
print(os.path.isfile("biblia.txt"))
print(os.path.isfile(os.path.join(os.getcwd(), "modulo_os/carpeta1/
archivo.txt")))
print(os.path.isfile("modulo_os"))
```

Salida

```
'Ruta/en/la/que/se/ejecuta/el/script/otra_carpeta'
```

→ REPORTE AUTOMATIZADO

Situación: Generar reportes semanales a partir de datos CSV.

Análisis: Se usa csv.writer() para exportar los resultados procesados.

Lecciones aprendidas: Los formatos abiertos facilitan el intercambio de información entre herramientas.

→ COMUNICACIÓN CON APIS

Situación: Recibir y guardar datos en formato JSON desde una API.

Análisis: Se usa json.load() y json.dump() para procesar la información.

Lecciones aprendidas: JSON es ideal para la interoperabilidad de datos en la web.

→ GESTIÓN DE ARCHIVOS EN U

Situación: Monitorear archivos generados automáticamente.

Análisis: El módulo os permite automatizar la verificación y movimiento de archivos.

Lecciones aprendidas: Python puede interactuar directamente con el sistema operativo.

→ CREAR Y ESCRIBIR UN ARCHIVO DE TEXTO

Objetivo: Generar un archivo nuevo con texto codificado.

Pasos:

1. Usar open('archivo.txt', 'w', encoding='utf-8').
2. Escribir contenido con .write() o .writelines().

3 Cerrar el archivo o usar with para hacerlo automáticamente. **Verificación:** Revisar el archivo en el explorador de archivos o con print(open('archivo.txt').read()).

→ LEER UN ARCHIVO CSV CON PANDAS

Objetivo: Visualizar datos en formato tabular.

Pasos:

1 Instalar pandas con pip install pandas.

2 Importar con import pandas as pd.

3 Usar pd.read_csv('archivo.csv').

4 Visualizar con .head().

ADVERTENCIA: Abrir un archivo en modo 'w' **sobrescribe** su contenido.
▶ **Prevención:** Usar el modo 'a' para agregar sin borrar.

ADVERTENCIA: I uso incorrecto de rutas absolutas puede causar errores.
▶ **Prevención:** Preferir rutas relativas y verificar con os.getcwd().

EJERCICIOS

01 Leer archivo TXT - Contar líneas
Crea un archivo datos.txt con varias líneas de texto y escribe un programa que cuente cuántas líneas tiene el archivo.

02 Escribir archivo TXT - Lista a archivo
Toma una lista de nombres ["Ana", "Carlos", "María", "Luis"] y escribe cada nombre en una línea diferente en un archivo nombres.txt.

03 CSV - Leer y procesar datos
Lee un archivo empleados.csv que tiene columnas nombre,salario,departamento y calcula el salario promedio de todos los empleados.

04 SV - Escribir datos
Crea un archivo productos.csv y escribe al menos 5 productos con sus precios y categorías.

05 JSON - Leer y modificar
Lee un archivo config.json que contiene configuración de una app, modifica el valor de "version" y guarda los cambios.

06 JSON - Crear desde diccionario
Toma un diccionario de Python que represente un libro (título, autor, año) y guárdalo como libro.json.

07 Módulo OS - Listar archivos
Lista todos los archivos con extensión .py en el directorio actual usando el módulo os.

08 Módulo OS - Información de archivos
Para un archivo específico, muestra su tamaño en bytes y la fecha de última modificación.

09 Módulo OS - Crear directorios
Crea una estructura de directorios proyecto/src y proyecto/docs usando os.makedirs().

10 Módulo OS - Recorrer árbol de directorios
Recorre recursivamente un directorio y muestra todos los archivos que tengan más de 1MB de tamaño.

 Todos los derechos reservados

MÓDULO 5: **MANEJO DE ERRORES Y EXCEPCIONES EN PYTHON**

En este módulo aprenderás por qué **manejar los errores correctamente** es una parte esencial de todo buen programa. Un código sin control de errores puede fallar fácilmente y detenerse sin avisar, mientras que uno bien diseñado puede **anticipar los problemas y seguir funcionando sin colapsar**. Primero verás los tipos de **errores más comunes** en Python:

- **Errores de sintaxis**, que ocurren cuando el código está mal escrito (por ejemplo, si olvidas los dos puntos en un if).
- **Errores de ejecución o excepciones**, que aparecen mientras el programa se está ejecutando (por ejemplo, dividir entre cero o acceder a una variable inexistente).
- **Errores lógicos**, que no detienen el programa, pero hacen que los resultados sean incorrectos.

Después aprenderás a usar la estructura **try-except**, que sirve para **“atrapar” los errores antes de que detengan el programa**. También conocerás las variantes **else** (para ejecutar algo si no hubo error) y **finally** (para ejecutar código sin importar si hubo error o no).

Verás cómo usar **excepciones genéricas** (como **Exception**, que captura cualquier tipo de error) y **excepciones específicas** (como **ZeroDivisionError, NameError o ValueError**), que te permiten manejar cada situación de manera diferente.

Además, aprenderás a **crear tus propias excepciones personalizadas** con la instrucción **raise**, útil cuando quieres mostrar mensajes de error claros al usuario (por ejemplo, “La cantidad debe ser positiva”).

Por último, conocerás la librería **logging**, que permite **registrar los errores y eventos del programa** en un archivo. Esto es especialmente útil cuando desarrollas proyectos grandes o en equipo, ya que te ayuda a revisar qué sucedió y cuándo ocurrió un fallo. Para cerrar el módulo, aplicarás todo lo aprendido en un **proyecto práctico de control de inventario**. Ahí pondrás en práctica validaciones, manejo de excepciones y registro de errores, creando un sistema mucho más **estable, seguro y profesional**.

 Todos los derechos reservados

TIPOS DE EXCEPCIONES

Puede revisar los códigos de formato en: https://docs.python.org/3/library/datetime.html#strftime-and-strptime-format-codes

→ ERROR DE SINTAXIS

Entrada

```
2valor = "nico"
```

Salida

```
SyntaxError: invalid decimal literal
```

Entrada

```
fur i in range(10)
    print(i)
```

Salida

```
SyntaxError: invalid syntax
```

Entrada

```
pront("Nicolás")
```

Salida

```
NameError: name 'pront' is not defined
```

→ ERROR DE EXCEPCION

Entrada

```
10 / 0
```

Salida

```
ZeroDivisionError: division by zero
```

Un número no se puede dividir por 0

Entrada

```
print(nombre)
```

Salida

```
NameError: name 'nombre' is not defined
```

La variable "nombre" no se ha definido

Entrada

```
lista = [1,2,3]
print(lista[1])
print(lista[4])
```

Salida

```
IndexError: list index out of range
```

La lista no tiene indice 4

→ ERROR DE LÓGICA

Entrada

```
def sumar(*args):
    """summary
    Función que returna la suma de todos los valores pasados como
argumentos
    Returns:
    suma: suma de los argumentos.
    """
    # return sum(args)
    return sum(args) + 1
```

```
# Error de lógica cuando se agrega el + 1. Mencionar cuando se hagan
pruebas unitarias esto generaría error.
sumar(2,45,8)
```

→ TRATAMIENTO DE ERRORES CON TRY-EXCEPT

Sintaxis:

try:
    Código que Python “intentará” ejecutar
except:
    Código que será ejecutado al capturarse una expreción
else:
    Código que se ejecuta si no se captura ninguna excepción (opcional)
finally:
    Bloque de código ejecutado siempre (opcional)

→ MANEJO GENERICO CON TRY-EXCEPT

Entrada

```
try:
    resultado = 10/0
    print("El resultado de la división es: ", resultado)
except:
    print("No es posible desarrollar esta división
(indeterminado).")

print("Programa continúa...")
```

Salida

```
No es posible desarrollar esta división (indeterminado).
Programa continúa...
```

Continúa en la próxima página

Entrada

```
try:
    print(nombre)
except:         # Añadir después el NameError
    print("Estás intentando imprimir una variable que no está
declarada ni asignada....")
```

Salida

```
Estás intentando imprimir una variable que no está declarada ni
asignada...
```

→ EXCEPCIÓN ESPECÍFICA

Entrada

```
try:
    resultado = 5 / 0  # Cambiar el orden de estas dos líneas.
    print(nombre)
except NameError:         # Añadir después el NameError
    print("Estás intentando imprimir una variable que no está
declarada ni asignada....")
except ZeroDivisionError:         # Añadir después el NameError
    print("No es posible realizar una división por 0
(indeterminado).")
```

Salida

```
No es posible realizar una división por 0 (indeterminado).
```

→ USO DE ELSE Y FINALLY

Si la sentencia no genera error se ejecuta el else y siempre se va a ejecutar el finally

Continúa en la próxima página

Entrada

```
try:
    resultado = 5 / 2
except ZeroDivisionError:
    print("Error: división por cero")
else:
    print(f"Resultado: {resultado}")
finally:
    print("Operación finalizada")
```

Salida

```
Resultado: 2.5
Operación finalizada
```

→ EXCEPCIONES PERSONALIZADAS CON RAISE

La palabra clave raise se usa para lanzar una excepción en el flujo de ejecución del programa. Puedes usarla para:

- Volver a lanzar una excepción **existente**.
- Lanzar una excepción **personalizada**.

Entrada

```
def dividir(a, b):
   if b == 0:
     raise ZeroDivisionError("Recuerda no dividir por cero.")
   #raise Exception("El divisor no puede ser igal a cero...")
    return a / b

dividir(234,0)
```

Continúa en la próxima página

Salida

```
ZeroDivisionError: Recuerda no dividir por cero!
```

Quite el comentario “#” dependiendo de qué mensaje quiera

→ REGISTRO DE ERRORES CON LOGGING

Entrada

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s",
    filename="eventos.log",
    filemode="a"
)

try:
    resultado = 10 / 0
except ZeroDivisionError:
    logging.error("División por cero.")
```

Salida

```
2025-11-19 10:54:14,084 - ERROR - División por cero. (- También el
archivo .log -)
```

→ CASO 1: ERROR LÓGICO EN FUNCIÓN MATEMÁTICA

Situación: Una función de suma devuelve resultados incorrectos.

Análisis: El código sumaba los valores pero añadía +1 al resultado final.

Lección: Los errores de lógica no generan mensajes de error, pero afectan la precisión del programa.

→ CASO 2: VALIDACIÓN DE ENTRADAS DE USUARIO

Situación: El usuario ingresa texto donde se espera un número.

Análisis: Uso de try-except para evitar la interrupción del programa.

Lección: Validar entradas protege la integridad de los datos y mejora la experiencia del usuario.

→ CASO 3: REGISTRO DE ERRORES EN APLICACIONES REALES

Situación: Un sistema web genera múltiples errores por valores nulos.

Análisis: Implementar logging permitió identificar los puntos críticos.

Lección: Registrar eventos facilita la depuración y mantenimiento.

→ PROCESO: CREAR UN SISTEMA DE MANEJO DE ERRORES

Objetivo: Prevenir fallos y mejorar la estabilidad del código.

Pasos:

1. Encierra el código crítico dentro de un bloque try.
2. Usa except para capturar errores específicos.
3. Añade else para acciones cuando no hay errores.
4. Utiliza finally para liberar recursos o mostrar mensajes finales.

Verificación: El programa debe continuar su ejecución incluso tras un error.

→ PROCESO: REGISTRAR ERRORES CON LOGGING

Objetivo: Guardar eventos del programa en un archivo .log.

Pasos:

1. Importar la librería: import logging.

2. Configurar con basicConfig() (nivel, formato, archivo).
3. Usar logging.error() o logging.warning() dentro de except.
4. Revisar el archivo .log generado.

Verificación: El programa debe continuar su ejecución incluso tras un error.

ADVERTENCIA: El uso de except: sin especificar el tipo de error puede ocultar problemas graves.
▶ **Prevención:** Siempre usa excepciones específicas (ValueError, TypeError, etc.).

ADVERTENCIA: Nunca uses raise sin contexto si no comprendes qué excepción genera.
▶ **Prevención:** Define mensajes claros al lanzar excepciones personalizadas.

EJERCICIOS

01 **Tipos de excepciones - División por cero**
Escribe un programa que pida dos números al usuario y realice la división, manejando específicamente el error de división por cero.

02 **try...except - Conversión segura**
Crea una función que convierta un input del usuario a entero, manejando el ValueError si el usuario ingresa algo que no es número.

03 **try...except múltiple - Acceso a lista**
Dada una lista [1, 2, 3], pide al usuario un índice y muestra el valor, manejando tanto IndexError como ValueError.

04 **try...except...else - Validación de archivo**
Intenta abrir un archivo datos.txt, maneja el FileNotFoundError, y si se abre correctamente, muestra su contenido usando else.

05 **try...except...finally - Conexión simulada**
Simula una conexión a base de datos que siempre debe cerrarse en el bloque finally, incluso si ocurre una excepción.

06 **raise - Validación de edad**
Crea una función que valide si una persona es mayor de edad (≥18 años), lanzando una excepción personalizada EdadInsuficienteError si no lo es.

07 **raise - Validación de contraseña**
Valida que una contraseña tenga al menos 8 caracteres, lanzando una excepción personalizada si no cumple el requisito.

08 **Excepciones personalizadas - Saldo insuficiente**
Crea una clase SaldoInsuficienteError que herede de Exception y úsala en una función de retiro bancario.

09 **logging - Registro de errores**
Crea un archivo datos.txt con varias líneas de texto y escribe un programa que cuente cuántas líneas tiene el archivo.

10 **logging - Manejo avanzado**

Modifica el ejercicio anterior para que los errores críticos se envíen por email (simulado) y los warnings se muestren en consola.

Bonus - Proyecto inventario

Implementa manejo de errores en un sistema de inventario: validar que las cantidades sean números positivos, que los productos existan antes de vender, etc.

MÓDULO 6: **PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN PYTHON**

En este módulo aprenderás uno de los temas más importantes y poderosos de la programación moderna: la **Programación Orientada a Objetos (POO).**

La POO es una forma diferente de pensar el código. Mientras en otros enfoques (como el **imperativo**, **funcional** o **declarativo**) se dan instrucciones paso a paso, en la POO el programa se organiza en "**objetos**", que representan cosas del mundo real, como una persona, un carro o un producto.

Cada objeto tiene sus **propiedades (atributos)** y sus **acciones (métodos)**. Por ejemplo:

- Un **carro** puede tener atributos como *color, marca y modelo*.
- Y métodos como *acelerar() o frenar()*.

De esta manera, la POO te permite escribir código **más ordenado, reutilizable y fácil de mantener.** Durante el módulo aprenderás los conceptos esenciales:

- **Clase:** es el molde o plano que define cómo serán los objetos.
- **Objeto (instancia):** es un ejemplo real creado a partir de una clase.
- **Atributos:** son las características o datos del objeto.
- **Métodos:** son las acciones o comportamientos que el objeto puede realizar.
- **Constructor (__init__):** es la función que se ejecuta automáticamente cuando creas un objeto.

Además, estudiarás los cuatro pilares de la POO, que son la base de este paradigma:

1. **Encapsulamiento:** protege los datos internos del objeto para que no sean modificados por error.
2. **Herencia:** permite crear nuevas clases a partir de otras ya existentes, reutilizando su código.
3. **Polimorfismo:** permite que distintas clases usen el mismo método, pero con comportamientos diferentes.
4. **Abstracción:** se enfoca en lo esencial, ocultando los detalles innecesarios al usuario.

Finalmente, aplicarás todos estos conceptos en un **proyecto práctico**, donde simularás una **carrera de autos**. En este ejercicio verás cómo los objetos (los autos) interactúan entre sí, cada uno con sus propias características y comportamientos, demostrando el poder y la utilidad de la POO en situaciones reales.

→ CREAR UNA CLASE SIMPLE

class Carro:

```
    pass
```

Explicación: Se declara una clase vacía usando class. La palabra clave pass evita errores de sintaxis.

→ CREAR OBJETOS (INSTANCIAS)

Entrada

```
mi_carro = Carro()
carro = Carro()

print(type(mi_carro))
print(type(carro))
```

Salida

```
<class '__main__.Carro'>
<class '__main__.Carro'>
```

→ CLASE CON CONSTRUCTOR Y ATRIBUTOS

Entrada

```
class Carro:
    def __init__(self, color, marca):
      self.color = color
      self.marca = marca
```

Instancia y ejecución

Entrada

```
mi_carro = Carro("gris", "Volkswagen")
carro = Carro("azul", "Nissan")

print(mi_carro.color)
print(carro.color)
```

Salida

```
gris
azul
```

→ MÉTODOS EN UNA CLASE

Entrada

```
class Carro: def init(self, color, marca): self.color = color
self.marca = marca
def hello_car(self):
    print(f"Hola {self.marca}, me gusta tu color {self.color}")

carro1 = Carro("rojo", "Ferrari")
carro1.hello_car()
```

Salida

```
Hola Ferrari, me gusta tu color rojo
```

 Todos los derechos reservados

→ CLASE CON MÚLTIPLES MÉTODOS

Entrada

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def presentarse(self):
        print(f"Hola, mi nombre es {self.nombre} y tengo {self.edad}
años.")

    def despedirse(self):
        print(f"Fue un gusto conocerte. Mi nombre es {self.nombre}.
¡Adiós!")
```

Instancia y ejecución

Entrada

```
p1 = Persona("Juan", 32)
p2 = Persona("Andrés", 35)

p1.presentarse()
p1.despedirse()
```

Salida

```
Hola, mi nombre es Juan y tengo 32 años.
Fue un gusto conocerte. Mi nombre es Juan. ¡Adiós!
```

→ ENCAPSULAMIENTO

El **encapsulamiento** está relacionado con la **seguridad y protección de los atributos** de una clase. La buena práctica dice que:

- Los atributos deben ser **privados.**
- Se deben modificar **solo a través de métodos.**

Problema en Python

En lenguajes como **Java** o **C#**, existen modificadores (public, private). En **Python todo es público**, pero usamos **convenciones**: La convención es de uno o dos guiones bajo (“_” o “__”)

Entrada

```
class Perro:
    def __init__(self, nombre):
        self.__nombre = nombre  # Atributo privado

    def get_nombre(self):
        return self.__nombre

    def set_nombre(self, nuevo_nombre):
        self.__nombre = nuevo_nombre

animal = Perro("Firulais")
animal.__nombre
```

Salida

```
AttributeError: 'Perro' object has no attribute '__nombre'
```

→ HERENCIA

La herencia es uno de los pilares de la POO. Consiste en crear una clase que hereda las características de otra clase. La clase que hereda se llama clase hija. La clase que transmite atributos y métodos se llama clase padre.

Entrada

```
class Animal:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def hablar(self):
        print("El animal hace un sonido.")

class Perro(Animal):
    def __init__(self, nombre, edad, raza):
        super().__init__(nombre, edad)
        self.raza = raza

    def hablar(self):
        print(f"{self.nombre} dice: Guau Guau")
```

➔ POLIMORFISMO

El **polimorfismo** es uno de los pilares de la POO. Aunque el nombre parezca complejo, el concepto es sencillo: 👉 Es la capacidad de una función o método de **comportarse de manera diferente dependiendo del tipo de dato** con el que trabaja. En Python usamos polimorfismo todo el tiempo, incluso sin darnos cuenta.

Entrada

```
class Vehiculo:
    def __init__(self, marca):
        self.marca = marca
    def mostrar_info(self):
        print(f"Vehículo de la marca {self.marca}")

class Coche(Vehiculo):
    def mostrar_info(self):
        print(f"Coche de la marca {self.marca}")

class Moto(Vehiculo):
```

```
        def mostrar_info(self):
            print(f"Moto de la marca {self.marca}")
```

Uso

Entrada

```
vehiculos = [Coche("Ferrari"), Moto("Yamaha")]
for v in vehiculos:
    v.mostrar_info()
```

- Cuando el bucle llega al objeto Coche("Ferrari"), llama a la versión de mostrar_info() definida en la clase Coche.
- Cuando llega al objeto Moto("Yamaha"), llama a la versión de mostrar_info() definida en la clase Moto.

El sistema de ejecución sabe en tiempo de ejecución qué versión del método debe llamar, basándose en el tipo real del objeto.

→ CASO 1: CLASE PERSONA

Situación: Representar personas con nombre y edad.

Análisis: Se usaron atributos para guardar información y métodos para definir acciones.

Lección: Las clases permiten modelar objetos del mundo real de forma clara y escalable.

→ CASO 2: HERENCIA ANIMAL–PERRO–GATO

Situación: Extender el comportamiento de una clase general (Animal) a clases específicas.

Análisis: super() permite heredar atributos y métodos base.

Lección: La herencia promueve la reutilización del código.

→ CASO 3: PROYECTO “CARRERA DE AUTOS”

Situación: Simular una carrera entre dos autos controlados por distintos conductores.

Análisis: Cada auto se modela como una clase con atributos (marca, color, distancia) y métodos (recorrer distancia).

Lección: La POO facilita la simulación de escenarios reales mediante objetos con comportamientos individuales.

→ PROCESO: CREAR UNA CLASE Y SUS OBJETOS

Objetivo: Aprender a instanciar objetos y usar sus métodos.

Pasos:

1. Definir la clase con **class**.
2. Implementar el constructor **__init__.**
3. Crear atributos y métodos.
3. Instanciar objetos con **objeto = Clase(parametros).**
4. Llamar a los métodos con **objeto.metodo().**

→ PROCESO: APLICAR ENCAPSULAMIENTO

Objetivo: Aprender a instanciar objetos y usar sus métodos.

Pasos:

1. Definir el atributo con doble guion bajo **__atributo.**
2. Crear un método **get_atributo()** para leerlo.
3. Crear un método **set_atributo()** para modificarlo.
4. Acceder únicamente a través de estos métodos.

 Todos los derechos reservados

→ PROCESO: APLICAR ENCAPSULAMIENTO

Objetivo: Aprender a instanciar objetos y usar sus métodos.

Pasos:

1. Definir la clase madre.
2. Crear la clase hija heredando entre paréntesis: **class Hija(Madre).**
3. Usar **super().__init__()** para heredar los atributos.
4. Agregar nuevos métodos o sobreescribir los heredados.

ADVERTENCIA: Sobrescribir métodos sin intención puede alterar el comportamiento de las clases hijas.
▶ **Prevención:** Revisa siempre los métodos heredados.

EJERCICIOS

01 **Clases y objetos - Persona**
Crea una clase Persona con atributos nombre, edad y nacionalidad, y un método presentarse() que muestre "Hola, soy {nombre}, tengo {edad} años".

02 **Encapsulamiento - Cuenta Bancaria**
Crea una clase CuentaBancaria con atributo privado __saldo y métodos públicos depositar() y retirar() que validen operaciones válidas.

03 **Abstracción - Figura Geométrica**
Crea una clase abstracta Figura con método abstracto calcular_area(), luego implementa clases Circulo y Rectangulo que hereden de ella.

04 **Herencia - Vehículos**
Crea una clase base Vehiculo con atributos marca y modelo, y luego clases Coche y Moto que hereden de ella añadiendo atributos específicos.

05 **Herencia múltiple - EstudianteTrabajador**
Crea clases Estudiante y Trabajador, y luego una clase EstudianteTrabajador que herede de ambas, combinando sus atributos y métodos.

06 **Polimorfismo - Animales**
Crea una clase Animal con método hacer_sonido(), y luego clases Perro, Gato y Vaca que sobrescriban este método con sonidos diferentes.

07 **Métodos especiales - Libro**
Implementa los métodos __str__, __eq__ y __len__ en una clase Libro para mostrar información, comparar por título y obtener páginas respectivamente.

08 **Propiedades - Temperatura**
Crea una clase Temperatura con propiedad celsius que automáticamente actualice su equivalente en fahrenheit cuando se modifique.

09 **Composición - Universidad**
Crea clases Aula, Profesor y Curso, donde Curso use composición para tener un aula y un profesor como atributos.

10 **Proyecto - Sistema de Carreras**
Crea clases CocheCarrera con atributos velocidad_maxima y aceleracion, y una clase Carrera que simule una competencia entre varios coches.

 Todos los derechos reservados

MÓDULO 7: **BASES DE DATOS EN PYTHON CON SQLITE**

En este módulo entrarás al fascinante mundo de las **bases de datos**, aprendiendo a usarlas de manera práctica con **SQLite**, una herramienta ligera y muy útil para comenzar a trabajar con información real.

Primero entenderás qué es una **base de datos**: un sistema que permite **guardar, organizar y administrar información** de forma estructurada. Por ejemplo, una base de datos puede almacenar la información de los estudiantes de un colegio, los productos de una tienda o los clientes de una empresa. Verás cómo los datos se organizan en una estructura compuesta por:

- **Tablas**, que son como hojas de cálculo.
- **Filas (registros)**, que representan cada entrada o elemento (por ejemplo, un estudiante o un producto).
- **Columnas (campos)**, que guardan los diferentes tipos de información (como nombre, edad o precio).

También aprenderás sobre los **DBMS (Database Management Systems)**, que son los programas que administran las bases de datos. Estos sistemas actúan como intermediarios entre tú (el usuario o programador) y los datos. En este módulo trabajarás con **SQLite**, un DBMS muy sencillo que ya viene integrado en Python.

A continuación, aprenderás a **instalar y usar SQLite Studio**, una herramienta visual que facilita la creación y gestión de bases de datos. Con ella podrás:

- Crear una nueva base de datos.
- Diseñar tablas.
- Insertar y consultar información.
- Probar comandos SQL sin necesidad de escribir mucho código.

Después verás cómo **conectar Python con SQLite** usando la librería **sqlite3**, que permite ejecutar desde tu código las mismas operaciones que harías en SQLite Studio. Practicarás los **comandos más importantes de SQL**, como:

- **CREATE** → para crear tablas.
- **INSERT** → para agregar datos.
- **SELECT** → para consultar información.
- **UPDATE** → para modificar registros.
- **DELETE** → para eliminar datos.

También aprenderás a manejar **transacciones** (grupos de operaciones que se ejecutan juntas) y a crear **vistas**, que son consultas guardadas para reutilizar después. El módulo termina mostrando cómo **eliminar estructuras completas**, como tablas o vistas, directamente desde el código en Python.

En resumen, al finalizar este módulo serás capaz de **crear, conectar y manipular bases de datos** desde Python, dominando las operaciones esenciales de SQL y comprendiendo cómo los programas almacenan y gestionan grandes cantidades de información.

→ PROCESO: INSTALAR Y CONFIGURAR SQLITE STUDIO

Objetivo: Tener una interfaz visual para administrar bases de datos.

Pasos:

1. Buscar “SQLite Studio” en el navegador.
2. Descargar la versión correspondiente al sistema operativo (Windows, macOS, Linux).
3. Ejecutar el instalador y aceptar los pasos del asistente.
4. Abrir la aplicación y crear una nueva base de datos con “Add Database”.

Verificación: El programa debe continuar su ejecución incluso tras un error.

Recuerde que todos los comandos se pueden ejecutar en SQLite y desde python para verificar que hacen lo mismo.

→ CREAR Y CONECTAR UNA BASE DE DATOS

Entrada

```
import sqlite3 # Siempre escribri
# Crear o conectar a la base de datos
connection = sqlite3.connect('curso.db') # siempre escribir
```

- Si el archivo cursos.db no existe, se crea automáticamente en la misma carpeta.
- Si ya existe, se establece conexión con él.

→ CREAR Y CONECTAR UNA BASE DE DATOS

Entrada

```
# Crear cursos (para ejecutar comandos SQL)
cursor = connection.cursor() # Siempre escribir

# Creando tabla con SQL
cursor.execute("""
CREATE TABLE alumno (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nombre VARCHAR(100) NOT NULL,
fecha_creacion DATE
);
""")

# Cerrar conexión
cursor.close()
```

→ INSERTAR DATOS

Entrada

```
# Ejecutar inserción
cursor.execute("""
INSERT INTO alumno (nombre, fecha_creacion)
VALUES ('Carlos', '2025-01-01');
""")
connection.commit()
```

→ CONSULTAR REGISTROS

Entrada

```
cursor.execute("SELECT * FROM alumno;")
datos = cursor.fetchall()
print(datos)
```

Salida

```
[(1, 'Carlos', '2025-01-01'), (2, 'Lucas', '2025-02-16'), (3,
'Gabriel', '2025-04-29'), (4, 'Maria', '2023-05-10')]
```

→FILTRAR REGISTROS

SELECT * FROM alumno WHERE id = 5;

→ ACTUALIZAR REGISTROS

Entrada

```
cursor.execute("""
UPDATE alumno
SET nombre = 'María José'
WHERE id = 4;
""")
connection.commit()
```

 Todos los derechos reservados

→ ELIMINAR REGISTROS

Entrada

```
# Eliminar registro con id = 2
cursor.execute("DELETE FROM alumno WHERE id = 2;")
connection.commit()
```

→CREAR UNA VISTA

```
CREATE VIEW vw_alumnos_2025 AS
SELECT * FROM alumno
WHERE strftime('%Y', fecha_creacion) = '2025';
```

→ EJEMPLO 10: CONSULTAR UNA VISTA DESDE PYTHON

Entrada

```
cursor.execute("SELECT * FROM vw_alumnos_2025;")
print(cursor.fetchall())
```

→ CASO 1: SISTEMA DE REGISTRO DE ESTUDIANTES

Situación: Crear una base de datos para almacenar información de alumnos.

Análisis: Se define una tabla alumno con campos **id, nombre y fecha_creacion.**

Lección: Las bases de datos permiten organizar y recuperar información de manera eficiente.

→ CASO 2: CONTROL DE VERSIONES DE DATOS

Situación: Se requiere guardar cambios y revertir operaciones erróneas.

Análisis: Uso de transacciones con **BEGIN, COMMIT y ROLLBACK.**

Lección: Garantiza la consistencia de los datos y evita pérdidas de información.

→ CASO 3: REPORTE ANUAL

Situación: Se necesita un reporte de alumnos creados en 2025.

Análisis: Se crea una **vista** para almacenar la consulta filtrada.

Lección: Las vistas simplifican el acceso a consultas frecuentes y complejas.

→ PROCESO: CREAR TABLA DESDE PYTHON

Objetivo: Generar una tabla relacional desde código.

Pasos:

1. Importar **sqlite3.**
2. Conectarse con **sqlite3.connect().**
3. Crear un cursor con **connection.cursor().**
4. Ejecutar la sentencia CREATE TABLE.
5. Confirmar con connection.commit().io que la tabla aparezca creada.

Verificación: Revisar en SQLite Studio que la tabla aparezca creada.

→ PROCESO: INSERTAR, CONSULTAR Y ACTUALIZAR REGISTROS

Objetivo: Generar una tabla relacional desde código.

Pasos:

1. Usar INSERT INTO para agregar registros.
2. Usar SELECT para consultarlos.
3. Usar UPDATE para modificarlos.
4. Confirmar cambios con c**ommit().**

Verificación: Consultar nuevamente la tabla para ver las actualizaciones.

 Todos los derechos reservados

ADVERTENCIA: Olvidar la cláusula WHERE en DELETE o UPDATE puede eliminar o modificar **toda la tabla.**

▶ **Prevención:** Siempre incluir una condición precisa.

ADVERTENCIA: No usar **commit()** después de insertar o actualizar puede **perder los cambios.**

▶ **Prevención:** Confirmar transacciones explícitamente.

ADVERTENCIA: Ejecutar múltiples sentencias SQL a la vez en **sqlite3** genera error.

▶ **Prevención:** Ejecutar cada sentencia por separado.

EJERCICIOS

01 **Crear base de datos y tabla - Estudiantes**
Crea una base de datos universidad.db con una tabla estudiantes que tenga: id (INTEGER PRIMARY KEY), nombre (TEXT), edad (INTEGER), carrera (TEXT).

02 **Insertar datos múltiples - Productos**
Crea una tabla productos e inserta al menos 5 productos diferentes con: id, nombre, precio, categoria.

03 **Consultas SELECT - Filtrar por condición**
De la tabla estudiantes, selecciona todos los estudiantes mayores de 20 años que estudien "Ingeniería".

04 **Actualizar datos - Aumento de precios**
Actualiza todos los productos de la categoría "Electrónicos" aumentando su precio en un 10%.

05 **Eliminar registros - Productos sin stock**
Elimina todos los productos cuyo precio sea 0 o que pertenezcan a la categoría "Descontinuado".

06 **Consultas con ORDER BY y LIMIT**
Selecciona los 3 productos más caros de la base de datos, ordenados de mayor a menor precio.

07 **Crear vista - Resumen de ventas**
Crea una vista resumen_productos que muestre por cada categoría: cantidad de productos y precio promedio.

08 **Eliminar tabla - Reestructuración**
Elimina una tabla empleados_temporal que ya no se necesita en la base de datos.

09 **Transacciones - Transferencia entre cuentas**
Simula una transferencia bancaria entre dos cuentas usando una transacción para asegurar que ambas operaciones (débito y crédito) se completen.

10 **Consulta JOIN - Estudiantes y calificaciones**
Crea dos tablas relacionadas: estudiantes y calificaciones, y realiza una consulta JOIN para mostrar el nombre del estudiante con sus notas.

MÓDULO 8: **INTRODUCCIÓN Y USO DE APIS EN PYTHON**

En este módulo aprenderás qué es una **API (Application Programming Interface)** y por qué son tan importantes en el mundo de la programación actual. Una **API** es básicamente un **puente de comunicación entre dos aplicaciones o servicios**. Imagina que tu programa necesita obtener el clima de una ciudad: en lugar de calcularlo tú mismo, puedes pedir esa información a una API meteorológica, que te la enviará lista para usar. Para entenderlo mejor, se explica cómo funciona la **interacción cliente-servidor:**

- El **cliente** (por ejemplo, tu programa) **envía una solicitud** al servidor.
- El **servidor** procesa esa solicitud y **responde con los datos o resultados** que el cliente necesita.

Este intercambio se realiza mediante **solicitudes (requests) y respuestas (responses)**, utilizando un protocolo llamado **HTTP**, el mismo que usa tu navegador web. A lo largo del módulo descubrirás las **ventajas y usos de las APIs** en distintos contextos, como:

- **Desarrollo web y móvil**, para conectar aplicaciones entre sí.
- **IoT (Internet de las cosas)**, donde los dispositivos se comunican automáticamente.
- **Automatización**, para enviar o recibir información sin intervención humana.
- **Ciencia de datos**, para acceder a datos en tiempo real desde internet.

También conocerás las **APIs REST**, el tipo más común hoy en día. Estas siguen ciertas reglas que facilitan la comunicación usando métodos HTTP, como:

- **GET →** para obtener información.
- **POST →** para enviar o crear nuevos datos.
- **PUT →** para actualizar información existente.

 Todos los derechos reservados

- **PATCH →** para modificar solo una parte.
- **DELETE →** para eliminar información.

Además, aprenderás a interpretar los **códigos de estado** que devuelve el servidor, como:

- **200 OK:** todo salió bien.
- **404 Not Found:** el recurso no existe.
- **500 Internal Server Error:** el servidor tuvo un problema.

Finalmente, pondrás en práctica todo lo aprendido realizando una **conexión real a una API pública** desde Python. Para ello, usarás las librerías **requests** (para enviar solicitudes) y **json** (para procesar las respuestas). De esta manera, verás cómo tu programa puede **conectarse a servicios externos, obtener información en tiempo real y trabajar con datos del mundo real.**

→ ESTRUCTURA BÁSICA CLIENTE–SERVIDOR

Contexto: Una aplicación web (cliente) solicita información meteorológica a un servidor remoto.

Desarrollo: El cliente envía una solicitud (request) al servidor del instituto meteorológico y este responde con datos climáticos (response).

Continúa en la próxima página

→ EJEMPLO 2: REALIZAR UNA SOLICITUD POST

Entrada

```
import requests, json
# URL de la API
url = https://api.restful-api.dev/objects
# Datos que se enviarán en la solicitud POST
payload = {
    "name": "MacBook Pro",
    "data": {"year": 2019, "price": 1849.99, "CPU model": "Intel
Core i9", "Hard disk size": "2 TB"}
}
# Encabezados de la solicitud
headers = {
"Content-Type": "application/json",
'Accept': 'application/json'}

try:
# Realizar la solicitud POST
  response = requests.post(url, data=json.dumps(payload),
headers=headers)
# Verificar el código de estado de la respuesta
    if response.status_code == 200:
        print("Objeto creado exitosamente")
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
                    print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

Salida

```
Objeto creado exitosamente:
{'id': 'ff808181932badb601963b87ce017f2f', 'name': 'Apple MacBook
Pro 16', 'createdAt': '2025-04-15T22:17:58.017+00:00', 'data':
{'year': 2019, 'price': 1849.99, 'CPU model': 'Intel Core i9', 'Hard
disk size': '2 TB'}}
```

 Todos los derechos reservados

→ SOLICITUD GET

Entrada

```
# Muestra los registros por defecto de la api, por eso no sale el
que creamos

# URL de la API con parámetros de consulta

url = https://api.restful-api.dev/objects

try:
    # Realizar la solicitud GET
    response = requests.get(url)

    # Verificar el código de estado de la respuesta
    if response.status_code == 200:
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
        print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

Salida

```
[{'id': '1', 'name': 'Google Pixel 6 Pro', 'data': {'color': 'Cloudy
White', 'capacity': '128 GB'}}, ...]
```

Continúa en la próxima página

→ GET CON PARÁMETROS

Entrada

```
# URL de la API con parámetros de consulta
url = "https://api.restful-api.dev/objects"
params = {
    'id': [id]  # Lista de IDs como parámetros de consulta
}

try:
    # Realizar la solicitud GET
    response = requests.get(url, params=params)

    # Verificar el código de estado de la respuesta
    if response.status_code == 200:
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
        print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

Salida

```
[{'id': 'ff808181932badb601963b8473cf7f2d', 'name': 'Apple MacBook
Pro 16', 'data': {'year': 2019, 'price': 1849.99, 'CPU model':
'Intel Core i9', 'Hard disk size': '2 TB'}}]
```

Continúa en la próxima página

→ GET CON PARÁMETROS

Entrada

```
# Método GET con Query Parameters

url = https://api.restful-api.dev/objects/
ff808181932badb601963b7d77cc7f29


try:
    response = requests.get(url, params=params)


    if response.status_code == 200:
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
        print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

Continúa en la próxima página

→ ACTUALIZAR DATOS (PUT)

Entrada

```
# URL de la API para la solicitud PUT
url = f"https://api.restful-api.dev/objects/{id}"

# Cuerpo de la solicitud PUT
payload = {
    "name": "Apple MacBook Pro 16",
    "data": {
        "year": 2019,
        "price": 1849.99,
        "CPU model": "Intel Core i9",
        "Hard disk size": "1 TB",
        "color": "Cloudy White",
        "capacity": "128 GB"
    }
}

# Encabezados de la solicitud
headers = {
    'Content-Type': 'application/json'
}

try:
    # Realizar la solicitud PUT
    response = requests.put(url, data=json.dumps(payload),
headers=headers)

    # Verificar el código de estado de la respuesta
    if response.status_code == 200:
        print("Datos actualizados exitosamente:")
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
        print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

Salida

```
Datos actualizados exitosamente:
{'id': 'ff808181932badb601963b8473cf7f2d', 'name': 'Apple MacBook
Pro 16', 'updatedAt': '2025-04-15T22:16:26.018+00:00', 'data':
{'year': 2019, 'price': 1849.99, 'CPU model': 'Intel Core i9', 'Hard
disk size': '1 TB', 'color': 'Cloudy White', 'capacity': '128 GB'}}
```

→ MODIFICACIÓN PARCIAL (PATCH)

Entrada

```
# URL de la API para la solicitud PUT
url = f"https://api.restful-api.dev/objects/{id}"

# Cuerpo de la solicitud PUT
payload = {
    "name": "Apple MacBook Pro 16 (update)"
}

# Encabezados de la solicitud
headers = {
    'Content-Type': 'application/json'
}

try:
    # Realizar la solicitud PUT
    response = requests.patch(url, data=json.dumps(payload),
headers=headers)

    # Verificar el código de estado de la respuesta
    if response.status_code == 200:
        print("Datos actualizados exitosamente:")
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
        print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

Salida

```
Datos actualizados exitosamente:
{'id': 'ff808181932badb601963b8473cf7f2d', 'name': 'Apple MacBook
Pro 16 (update)', 'updatedAt': '2025-04-15T22:16:47.599+00:00',
'data': {'year': 2019, 'price': 1849.99, 'CPU model': 'Intel Core
i9', 'Hard disk size': '1 TB', 'color': 'Cloudy White', 'capacity':
'128 GB'}}
```

→ ELIMINAR RECURSO (DELETE)

Entrada

```
# URL de la API con parámetros de consulta
url = f"https://api.restful-api.dev/objects/{id}"


try:
    # Realizar la solicitud DELETE
    response = requests.delete(url)

    # Verificar el código de estado de la respuesta
    if response.status_code == 200:
        print(response.json())
    else:
        print(f"Error: {response.status_code}")
        print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error en la solicitud: {e}")
```

→ CASO 1: APLICACIÓN DEL CLIMA

Situación: Una app muestra información meteorológica actualizada.

Análisis: Consulta una API pública de clima y muestra datos en tiempo real.

Lección: Las APIs permiten integrar servicios externos sin desarrollar funcionalidades desde cero.

→ CASO 2: AUTOMATIZACIÓN DE TAREAS

Situación: Una empresa automatiza el envío de reportes diarios.

Análisis: Se usa una API para comunicarse con un servicio de correo electrónico.

Lección: Las APIs reducen la intervención humana al automatizar procesos rutinarios.

→ CASO 3: INTEGRACIÓN DE IA

Situación: Una app financiera utiliza un modelo predictivo externo.

Análisis: El modelo se consume mediante una API de aprendizaje automático.

Lección: Las APIs facilitan el acceso a inteligencia artificial y recursos avanzados.

→ PROCESO: REALIZAR CONEXIÓN CON UNA API

Objetivo: Enviar y recibir datos entre Python y un servidor web.

Pasos:

1. Importar **requests y json.**
2. Definir la **URL del endpoint.**
3. Crear un **payload** con los datos a enviar (diccionario).
4. Especificar los **headers (Content-Type: application/json).**
5. Realizar la solicitud con el método adecuado **(get, post, etc.).**
6. Evaluar **response.status_code y procesar response.json().**

Verificación: Recibir respuesta con código **200 OK** y contenido JSON válido.

Continúa en la próxima página

ADVERTENCIA: Revisar siempre los códigos de estado antes de procesar la respuesta.
▶ **Prevención:** Implementar validaciones y manejo de errores (try-except).

ADVERTENCIA: Las APIs públicas pueden tener límites de uso (rate limits).
▶ **Prevención:** Consulta la documentación de cada API antes de automatizar consultas.

ADVERTENCIA: No envíes información sensible en URLs ni en cuerpo de solicitudes sin cifrado (HTTPS).
▶ **Prevención:** Usa siempre conexiones seguras y tokens de autenticación.

EJERCICIOS

01 **GET básico - Obtener datos de usuarios**
Realiza una petición GET a JSONPlaceholder API (https://jsonplaceholder.typicode.com/users) y muestra los nombres de todos los usuarios.

02 **GET con parámetros - Filtrar posts**
Realiza una petición GET a JSONPlaceholder para obtener solo los posts de un usuario específico (usando ?userId=1).

03 **POST - Crear nuevo recurso**
Realiza una petición POST a JSONPlaceholder para crear un nuevo post con título, cuerpo y userId, y muestra la respuesta.

04 **PUT - Actualizar recurso completo**
Realiza una petición PUT para actualizar completamente un post existente (ej: post con id 1) con nuevos datos.

05 **PATCH - Actualización parcial**
Realiza una petición PATCH para actualizar solo el título de un post existente, manteniendo los demás campos.

06 **DELETE - Eliminar recurso**
Realiza una petición DELETE para eliminar un post específico y verifica el código de respuesta.

07 **Manejo de headers - Autenticación básica**
Realiza una petición a una API que requiera headers de autenticación (puedes usar una API de prueba como reqres.in).

08 **Manejo de errores - Códigos de estado**
Crea una función que maneje diferentes códigos de estado HTTP (200, 404, 500) mostrando mensajes apropiados.

09 **API con parámetros complejos - Consulta climática**
Usa una API de clima como OpenWeatherMap (o una alternativa gratuita) para obtener el clima actual de una ciudad.

10 **Proyecto integrador - Cliente API**
Crea una clase APIClient que encapsule métodos para GET, POST, PUT, PATCH y DELETE con manejo de errores.

MÓDULO 9: **INTRODUCCIÓN PRÁCTICA A STREAMLIT**

En este módulo descubrirás **Streamlit**, una biblioteca moderna y de código abierto que permite crear **aplicaciones web con Python** de forma rápida y sencilla, **sin necesidad de aprender HTML, CSS o JavaScript**. Streamlit te permite **convertir tus scripts de Python en aplicaciones interactivas** en cuestión de minutos.

Su gran ventaja es que tiene una **curva de aprendizaje baja**, por lo que es ideal para estudiantes, analistas de datos y programadores que quieren mostrar sus proyectos de manera visual. Además, se integra perfectamente con librerías como **Pandas** (para manejar datos) y herramientas de **visualización**.

Nota: Recuerde descargar la base de datos para trabajar.

Primero debes **preparar tu entorno profesional** usando **Poetry**, una herramienta que te ayudará a crear y activar tu entorno virtual (virtualenv) e instalar las dependencias necesarias de forma ordenada (Recuerde revisar el Módulo 1).

Luego ejecutarás tu primera aplicación de Streamlit en un **servidor local** (en la dirección localhost:8501). También aprenderás a **configurar la página**, personalizando el **título** y el **ícono** que se muestran en el navegador. Después, construirás **interfaces interactivas** usando los **widgets** que ofrece Streamlit, como:

- **st.text_input →** para capturar texto.
- **st.number_input →** para ingresar números.
- **st.date_input →** para seleccionar fechas (con formato personalizado).
- **st.selectbox y st.multiselect →** para elegir una o varias opciones.
- **st.radio →** para seleccionar una sola opción entre varias.
- **st.checkbox →** para activar o desactivar una condición.
- **st.button →** para ejecutar acciones al hacer clic.

A continuación, aprenderás a **trabajar con datos dentro de tu app**, por ejemplo:

- **Cargar archivos CSV.**
- **Mostrar y editar tablas con st.dataframe.**
- **Exportar la información nuevamente a un archivo CSV.**

También explorarás cómo mejorar el diseño visual mediante el **layout responsivo**, ajustando **columnas, bordes, alineaciones** y **el ancho de la página** para que la app se vea profesional.

Finalmente, conocerás cómo crear **aplicaciones multipágina** usando la carpeta pages/ y cómo aprovechar el **Session State**, una herramienta que permite **guardar y compartir información entre diferentes páginas** de tu aplicación sin perder los datos al cambiar de vista.

En resumen, este módulo te mostrará cómo pasar de un simple script de Python a una **aplicación web completa e interactiva**, perfecta para presentar tus proyectos, análisis o reportes de una manera moderna, visual y profesional.

→ PRIMERA APP Y SERVIDOR LOCAL

Contexto: Ejecutar una app básica y visualizar un DataFrame.

Desarrollo: Carga CSV con **pd.read_csv, ejecuta streamlit run app.py**, se abre **localhost:8501** (URL que se mostrará en la terminal y se debe copiar y pegar).

Entrada

```
import streamlit as st
import pandas as pd
# INTERFAZ DE LA APLICACIÓN
st.title("Mi primera app con Streamlit 😀")
# CARGA DE DATOS
# - Lee archivo CSV con datos de accidentes
# - Muestra tabla interactiva automáticamente
datos = pd.read_csv("acidentes.csv")
datos
```

→ WIDGETS DE TEXTO Y MARKDOWN

Contexto: Añadir **st.title, st.header, st.subheader, st.markdown** y bloques de código.

Desarrollo: Uso de encabezados jerárquicos, listas y bloque **python.**

Entrada

```
import streamlit as st
# ELEMENTOS DE TEXTO Y FORMATO
st.title("Título de mi primera app!")
st.write("Texto utilizando write :smile:")  # Soporta emojis y
markdown
st.text("Texto utilizando text.")   # Texto plano sin formato
# SECCIONES Y ESTRUCTURA
st.header("Sección")
st.subheader("Sub-Sección")
# MARKDOWN AVANZADO
st.header("Segunda Sección")
st.text("Texto utilizando text.")
st.header("Markdown")
st.markdown(""" # Título de primer nivel.
    ## Título de segundo nivel.
    ### Título de tercer nivel.
    - Elemento 1
    - Elemento 2
    - Elemento 3
    1. hola

    ```python
    nombre = "Nicolás"
    print(nombre)
    ```
    """)

# BLOQUES DE CÓDIGO
st.header("Códigos")
st.code("""
                        nombre = "Nicolás"
                        print(nombre)
                """)
```

Resumen de funciones usadas:

- **title():** Título principal
- **write():** Texto con formato y emojis
- **header()/subheader():** Organización por secciones
- **markdown():** Texto con formato avanzado
- **code():** Bloques de código con sintaxis

→ CAPTURA DE TEXTO CON RESTRICCIONES

Contexto: st.text_input con valor por defecto y límite de caracteres.

Desarrollo: Parámetros **value="Pepito", key="input_text", max_chars=10.**

→ NÚMERO CON RANGO Y PASO

Contexto: st.number_input con límites y **step=2.**

Desarrollo: Definir **min_value=0, max_value=10, step=2.**

→ FECHA CON FORMATO LOCAL

Contexto: ost.date_input y visualización “DD/MM/YYYY”.

Desarrollo: Usar **format** para el widget y **date.strftime("%d/%m/%Y")** para mostrar.

→ RADIO CON INDEX

Contexto: Elegir marca de auto con opción preseleccionada.

Desarrollo: st.radio(options=["Audi","Ferrari","Porsche"], index=1).

→ CHECKBOX DE TÉRMINOS

Contexto: Aceptación y formulario condicionado por **True/False.**

Desarrollo: Si **aceptar** es **True** → pedir **name, age, date** y botón con icono.

→ INSERTA MEDIA

Contexto: Añadir **imagen, audio** y un **video** de YouTube.

Desarrollo: st.image(..., width=...), st.audio(...), st.video("URL").

→ EDITAR Y EXPORTAR DATOS

Contexto: Editar tabla y guardar cambios.

Desarrollo: Usar **st.dataframe** editable y un botón “Guardar” que exporta a CSV.

Resultado/Conclusión: Se crea/reescribe archivo CSV sin columna de índice.

Proceso: Preparar entorno con Poetry

Objetivo: Aislar dependencias y registrar librerías del proyecto.

Pasos numerados:

1. Abrir VS Code y la carpeta del proyecto.
2. Inicializar proyecto: **poetry init** y aceptar valores por defecto; definir autor.
3. Crear/activar venv: **poetry shell** (instalar plugin **poetry-plugin-shell** si es necesario).
4. Instalar dependencias: **poetry add streamlit** (Pandas puede instalarse como dependencia colateral).

Verificación: Ver **pyproject.toml** con **streamlit** y pandas listados.

Proceso: Ejecutar la primera app.

Objetivo: Levantar el servidor local y visualizar datos.

Pasos numerados:

1. Crear **app.py**, importar **streamlit as st y pandas as pd.**
2. Leer CSV con **pd.read_csv(...)** y mostrarlo con **st.dataframe(...).**
3. Ejecutar **streamlit run app.py.**
4. Abrir **http://localhost:8501** (se abre automáticamente).

Verificación: Se visualiza tabla con búsqueda, descarga y cambio de tema.

Proceso: Configurar la página (título e icono)

Objetivo: Personalizar pestaña del navegador y favicon.

Pasos numerados:

1. Agregar **st.set_page_config(page_title="Mi app", page_icon="🚀").**

Verificación: Ver texto e icono actualizados en la pestaña.

Proceso: Formulario condicionado por checkbox

Objetivo: Mostrar campos solo si se aceptan términos.

Pasos numerados:

1. **aceptar = st.checkbox("Acepta términos y condiciones").**
2. **if aceptar:** mostrar **st.text_input, st.number_input, st.date_input(format="DD/MM/YYYY") y st.button(icon="...").**

Verificación: Los campos aparecen solo cuando el checkbox está marcado.

Procedimiento: Cargar datos con caché y mostrarlos.

Prerrequisitos: CSV en el directorio del proyecto.

Secuencia:

1. Paso 1: Decorar función de carga con @st.cache_data.
2. Paso 2: data = cargar_datos() y st.dataframe(data).

Puntos de control: DataFrame visible; recargas posteriores no releen el archivo.

Procedimiento: Selectores de opciones

Prerrequisitos: Lista de opciones definidas.

Secuencia:

1. Paso 1: **color = st.selectbox("Escoge un color", ["Verde","Azul","Rojo","Lila"], key="select_color").**
2. Paso 2: **colors = st.multiselect("Escoge los colores", ["Verde","Azul","Rojo","Lila"], key="multi_colors").**

Puntos de control: selectbox → string; **multiselect →** lista.

LIMITACIONES PARA PROYECTOS GRANDES

▶ **Consecuencia:** Falta de flexibilidad en layout/widgets; recargas en ciertas interacciones.

▶ **Prevención:** Para escalas complejas, evaluar frameworks web tradicionales.

RECARGA DE PÁGINA POR INTERACCIÓN

▶ **Consecuencia:** Sensación de “refresh” frecuente.

▶ **Prevención:** Diseñar flujos con cambios mínimos por interacción.

CAMPOS ACTIVOS SIN VALIDACIÓN

▶ **Consecuencia:** Formularios enviados con datos inválidos.

▶ **Prevención:** Usar límites/formatos y condiciones (checkbox antes de inputs).

EJERCICIOS

01 **Primera aplicación - Calculadora simple**
Crea una app Streamlit con dos inputs numéricos y botones para realizar operaciones básicas (suma, resta, multiplicación, división).

02 **Elementos de texto - Generador de CV**
Crea un formulario con text_input, text_area, number_input y date_input para que usuarios ingresen sus datos personales y muestre un CV formateado.

03 **Elementos de entrada - Sistema de registro**
Usa selectbox, multiselect, radio y checkbox para crear un formulario de registro de intereses con categorías y preferencias.

04 **Elementos de entrada avanzados - Uploader**
Implementa un file_uploader para cargar archivos CSV y muestra un preview del DataFrame usando pandas.

05 **Elementos de estado - Contador interactivo**
Crea un contador que use session_state para mantener el estado entre interacciones, con botones para incrementar/decrementar.

06 **Análisis de datos - Visualizador CSV**
Desarrolla una app que permita subir un CSV, seleccionar columnas y mostrar gráficos (línea, barras, dispersión) con plotly o matplotlib.

07 **Elementos de medios - Galería de imágenes**
Crea una galería que permita subir múltiples imágenes y mostrarlas en un layout de columnas con st.columns.

08 **Personalización de layout - Dashboard**
Usa sidebar, columns y expander para crear un dashboard organizado con métricas, gráficos y controles en la barra lateral.

09 **Layout avanzado - Tabs y containers**
Implementa una app con pestañas (tabs) para diferentes secciones: datos, análisis, visualizaciones, usando también container para agrupar elementos.

10 **Proyecto integrador - Sistema de inventario**
Crea una app completa para gestionar inventario: agregar productos, ver stock, actualizar precios, con base de datos SQLite y validaciones.

 Todos los derechos reservados

Daxus
LATAM

www.daxus.com

ESTE CONTENIDO ES PARTE INTEGRANTE DEL ENTRENAMIENTO RECURSOS AVANZADOS DEL LENGUAJE PYTHON
Y ES PROPIEDAD DE DAXUS LATAM. SE **PROHÍBE** LA COMERCIALIZACIÓN Y REPRODUCCIÓN TOTAL O PARCIAL.